



北京航空航天大学
BEIHANG UNIVERSITY



操作系统

王良

lwang20@buaa.edu.cn

■ 英文: Bootstrapping (简称boot)

➤ 它来自一句谚语: “pull oneself up by one's bootstraps”(据考证来自于《吹牛大王历险记》) 字面意思是"拽着鞋带把自己拉起来", 这当然是不可能的事情

➤ 最早的时候, 工程师们用它来比喻, 计算机启动是一个很矛盾的过程: 必须先运行程序, 然后计算机才能启动, 但是计算机不启动就无法运行程序!

- 早期必须想尽各种办法, 把一小段程序装进内存, 然后计算机才能正常运行。工程师们把这个过程叫做"拉鞋带", 久而久之就简称为boot了

OPERATION BOOTSTRAP





启动，是一个很“纠结”的过程

- 现代计算机 —— 硬件 + 软件
- 计算机功能的多样性和灵活性 vs 启动状态的单一性
 - 一方面：必须通过程序控制使得计算机进入特定工作状态
(必须运行启动程序来启动计算机)
 - 另一方面：程序必须运行在设置好工作模式的硬件环境上
(启动程序必须运行在启动好的计算机上)
- 因此：启动前硬件状态必须假设在一个最安全、通用，因此也是功能最弱的状态，需要逐步设置硬件，以提升硬件环境能力
- OS启动是一个逐步释放系统灵活性的过程



计算机的启动过程

- 操作系统作为一个可执行程序（由C语言和汇编语言编写生成），需要交给操作系统使用
- 硬件如何知道内核的位置或如何加载内核？
- 在大多数计算机系统中，包括一段小程序，称为**bootloader**（或称作bootstrap program），负责找到内核，将其加载到主存储器中，并启动其执行



计算机的启动过程

- CPU通电启动或者Reset时，CPU从一个预定义的内存位置开始执行bootloader
- bootloader程序一般存储于ROM只读存储器，因为RAM在系统启动时处于未知状态
 - ROM它不需要初始化，也不容易被计算机病毒感染
- 通过bootloader，我们可以**初始化硬件设备、建立内存空间的映射表**，从而建立适当的系统软硬件环境，为最终调用操作系统内核做好准备



Bootloader

■ Bootloader 需要正确地找到内核并加载执行，分为 stage1 和 stage2 两个部分

■ Stage 1

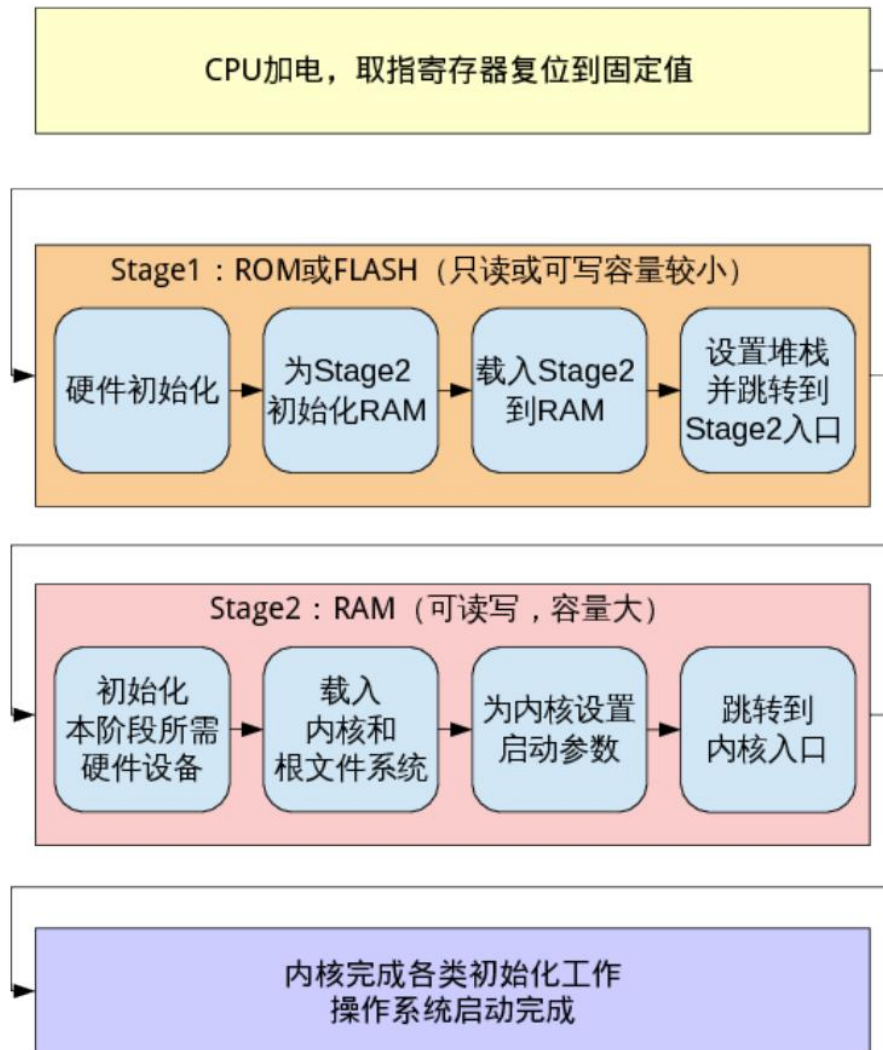
- 依赖于CPU体系结构的代码（如设备初始化代码等）通常都放在 stage1 且可以用汇编语言来实现
- 需要初始化硬件设备，包括 watchdog timer、中断、时钟、内存等
- bootloader 程序直接从非易失存储器上（比如 ROM 或 FLASH）加载，为加载 stage2 准备 RAM 空间

■ Stage 2

- 通常用C语言来实现，可实现复杂功能，更好的可读性和移植性
- 初始化这一阶段需要使用的硬件设备以及其他功能
- 将内核镜像从存储器读到 RAM 中，并为内核设置启动参数
- 将 CPU 指令寄存器的内容设置为内核入口函数的地址，即可将控制权从 bootloader 转交给操作系统内核



Bootloader





嵌入式系统Bootloader —— Uboot

■ U-Boot : 通用引导加载程序 (Universal Boot Loader)

- 开源、跨平台: 支持多种处理器架构 (ARM, MIPS, PowerPC, x86 等)
- 主要用于嵌入式系统: 路由器、开发板、网络设备、工业控制等



■ U-Boot 的主要功能

- 硬件初始化: CPU、内存、外设 (串口、Flash、网卡等)
- 引导流程控制: 支持从Flash、SD卡、网络 (TFTP) 加载内核
- 交互式命令行: 提供类似Shell的环境, 支持命令配置与调试

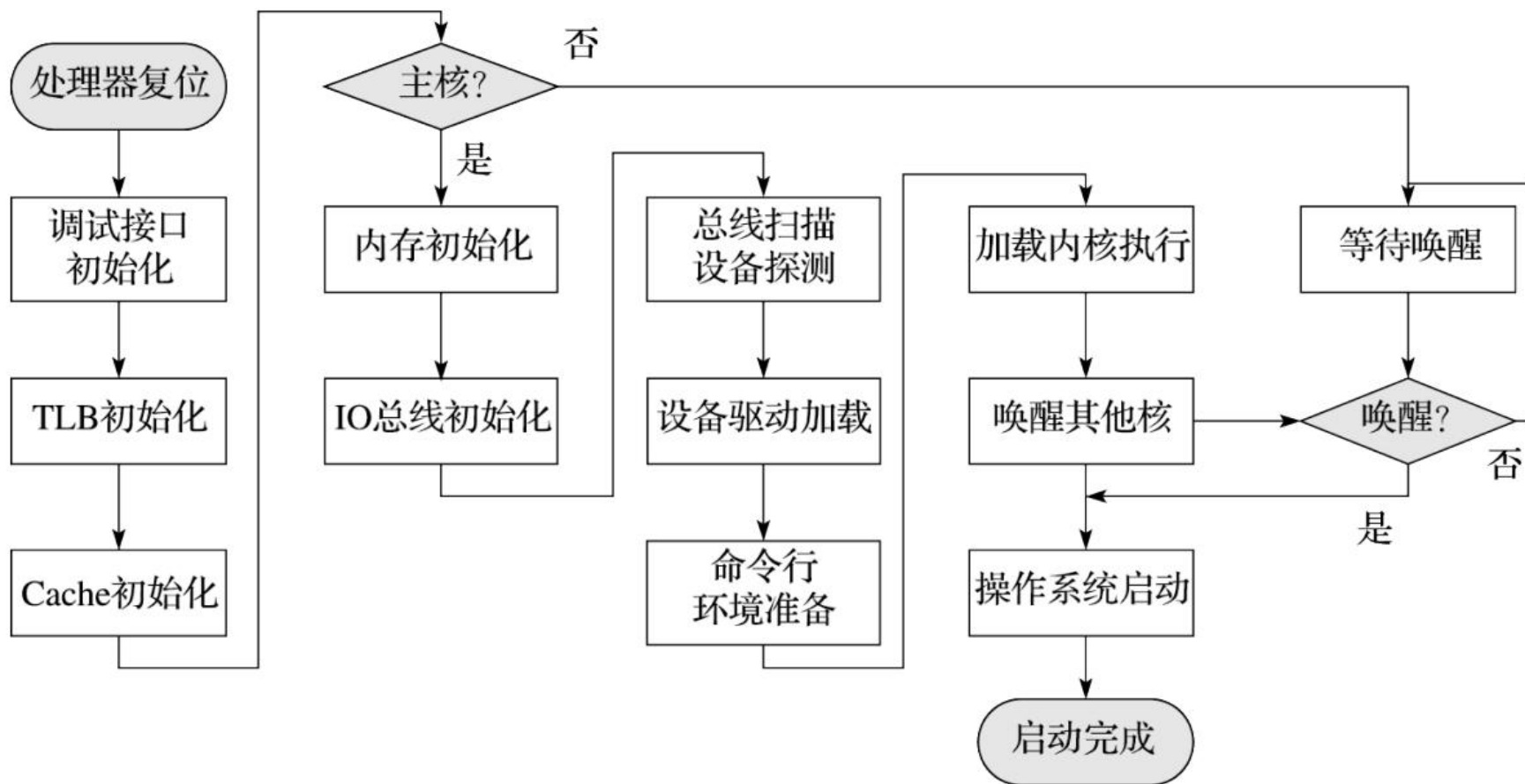


计算机的启动过程

- **一些小型系统（或嵌入式系统）将整个操作系统在ROM中**
 - 将操作系统存储在ROM中适用于小型操作系统及简单的硬件
 - 更改bootloader代码需要改变ROM硬件芯片，一些系统通过使用可擦除可编程只读存储器（EPROM或FLASH）
 - 容量小，价格贵，速度比RAM慢
- **对于大型操作系统（包括大多数通用操作系统，如Windows、Mac OS、Linux等），bootloader放在一般非易失存储器（ROM 或 FLASH）中，而将操作系统内核放在磁盘中，硬件启动和软件启动的分离**



计算机的启动过程



《计算机体系结构基础 第3版》，胡伟武等，2021



初始化过程

- **初始化实际是将计算机内部的各种寄存器状态从不确定设置为确定，将一些模块状态从无序强制为有序的过程**
- **处理器核初始化：对相关的寄存器内容进行初始化**
 - 处理器复位
 - 调试接口初始化
 - TLB 初始化
 - Cache 初始化
- **总线接口初始化**
 - 内存初始化
 - IO 总线初始化
- **设备的探测及驱动加载**

《计算机体系结构基础 第3版》，胡伟武等，2021



初始化过程

■ TLB 初始化

- 地址映射的管理模块，主要负责操作系统里用户进程地址空间的管理
- 将全部表项初始化为无效项，即将 TLB 的每项逐一清空，以免程序中使用的地址被未初始化的 TLB 表项所错误映射

■ Cache 初始化

- Cache 的组织结构主要包含标签 (Tag) 和数据 (Data) 两个部分
- 对 Cache 的初始化就是对 Tag 的初始化，只要将其中的 Cache 块状态设置为无效，其他部分的随机数据就不会产生影响
- 如果不经初始化，对于 Cache 空间的访问也可能会导致错误的命中

- 片上 Cache 的容量越来越大，初始化时间越来越长。因此，TLB、Cache 等结构的初始化也更多地开始使用硬件自动完成，软件在这些初始化上耗费的时间也越来越少



启动及OS引导

当我们打开计算机的电源开关，到我们看到OS的登录界面，计算机内部经历了怎样的过程？

■ MIPS CPU系统的启动

■ X86 CPU系统的启动

- 硬件初始化过程 (X86)
- Linux操作系统引导

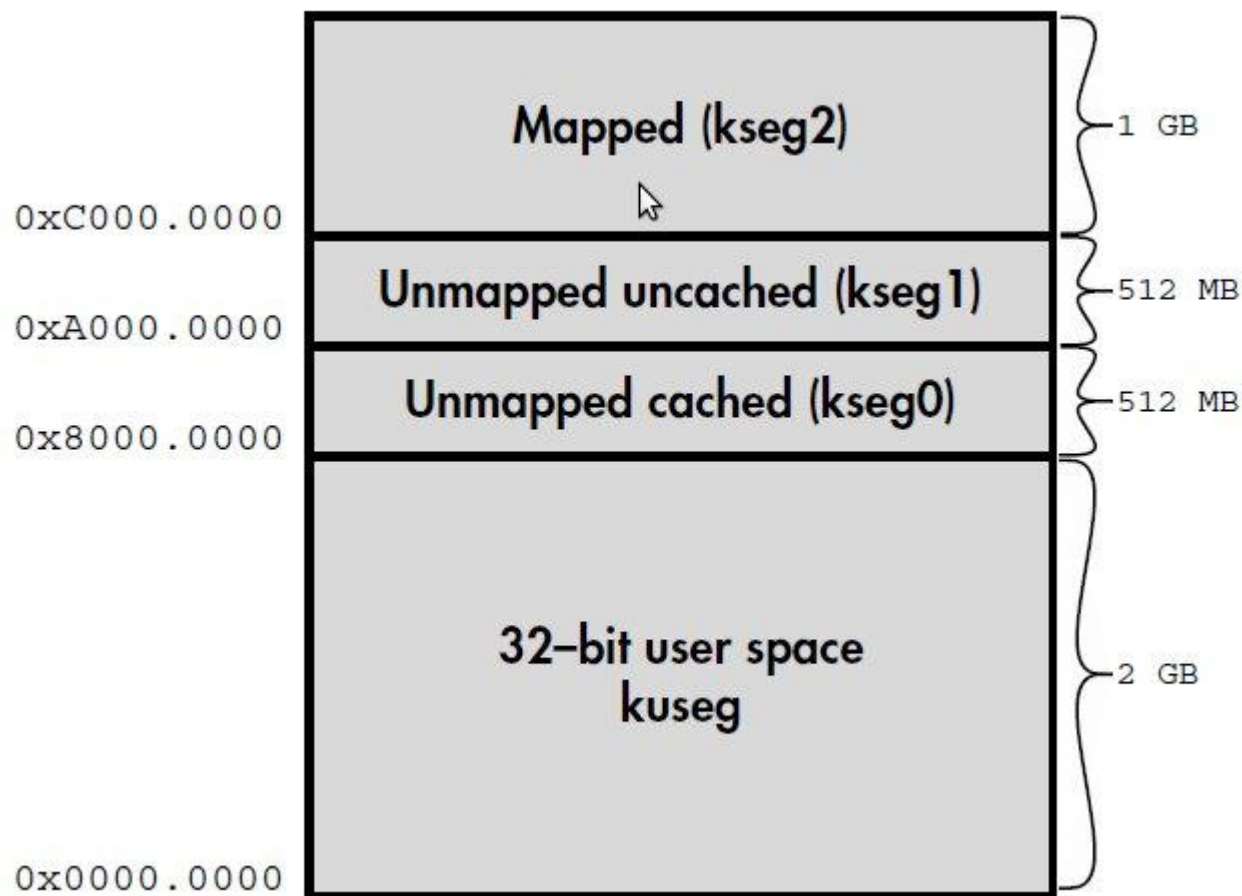
MIPS处理器大多用于嵌入式系统，嵌入式系统常用U-boot作为OS启动装载程序，U-Boot(Universal Boot Loader)

X86处理器通常采用LILO和GRUB



MIPS的基本地址空间

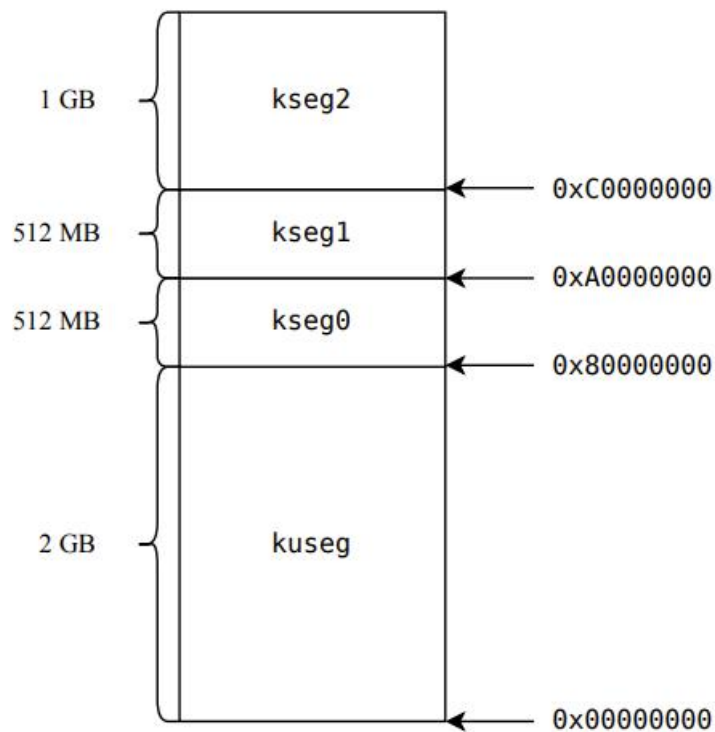
在32位下,程序地址空间(4G)划分为四大区域,不同区域有不同的属性





MIPS的基本地址空间

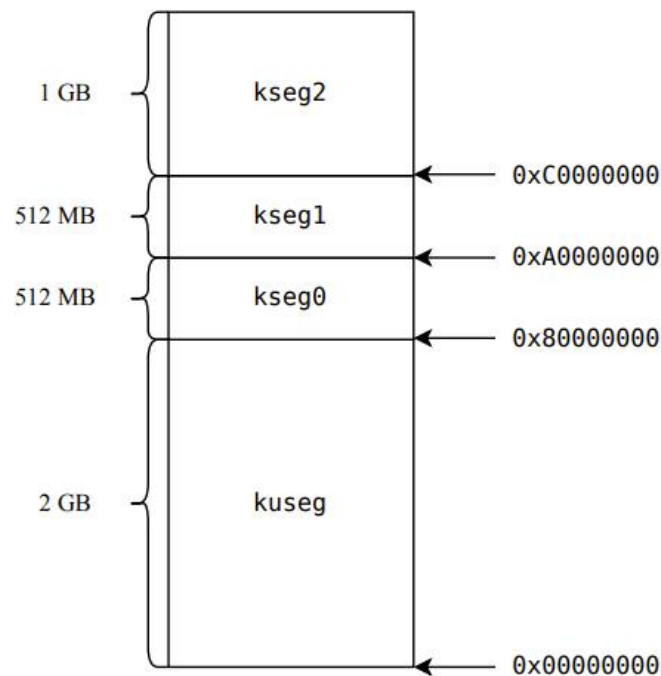
- **kuseg**: 这一段是用户态下唯一可用的地址空间（内核态下也可使用这段地址空间），大小为 2 GB，也就是 MIPS 约定的用户内存空间。需要通过MMU（Memory Management Unit）中的TLB 进行虚拟地址到物理地址的变换。对这段地址的存取都会通过 cache
- **kseg0**: 这一段是内核态下可用的地址，MMU 将地址的最高位清零 (& 0x7fffffff) 就得到物理地址用于访存。也就是说，这段的虚拟地址被连续地映射到物理地址的低 512 MB 空间。对这段地址的存取都会通过 cache





MIPS的基本地址空间

- **kseg1**: 将这些地址的高三位清零可映射到相应的物理地址上，这段虚拟地址也被连续地映射到物理地址的低 512 MB 空间。与kseg0映射的物理地址一样，但kseg1是非cache存取的。**kseg1是唯一在系统重启时能正常工作的地址空间**
- **kseg2**: 这段地址只能在内核态下使用并且需要 MMU 中TLB 将虚拟地址转换为物理地址。对这段地址的存取都会通过 cache



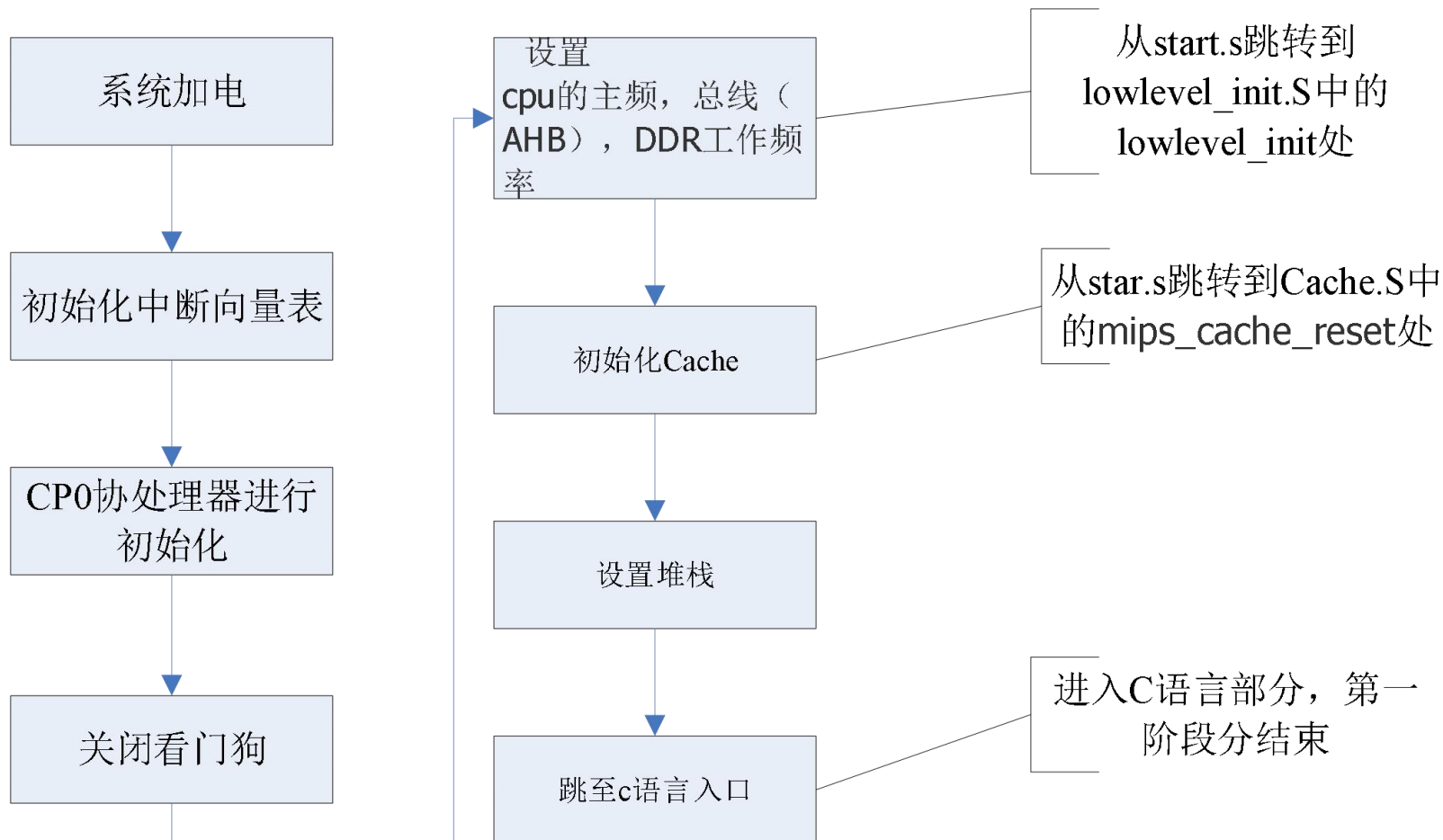


MIPS ROM/Flash启动地址

- MIPS上电启动时，由于OS尚未接管系统，不能采用TLB、Cache机制。从MIPS的初始内存划分可知，kseg1是唯一的在系统重启时能正常工作的内存映射地址空间
- MIPS的启动入口地址是0xBFC00000，通过将最高3位清零（&0x1fffffff）的方法，将ROM所在的地址区映射到物理内存的低端512M(0x00000000 - 0x1FFFFFFF)空间，也是“非翻译无需转换的”（Unmapped）地址区域
- 因此，kseg1是唯一的在系统重启时能正常工作的内存映射地址空间，这也是为什么重新启动时的入口向量是（0xBFC00000）会在这个区域。这个向量对应的物理地址是0x1FC00000



MIPS启动过程1





MIPS启动过程2 (C代码)

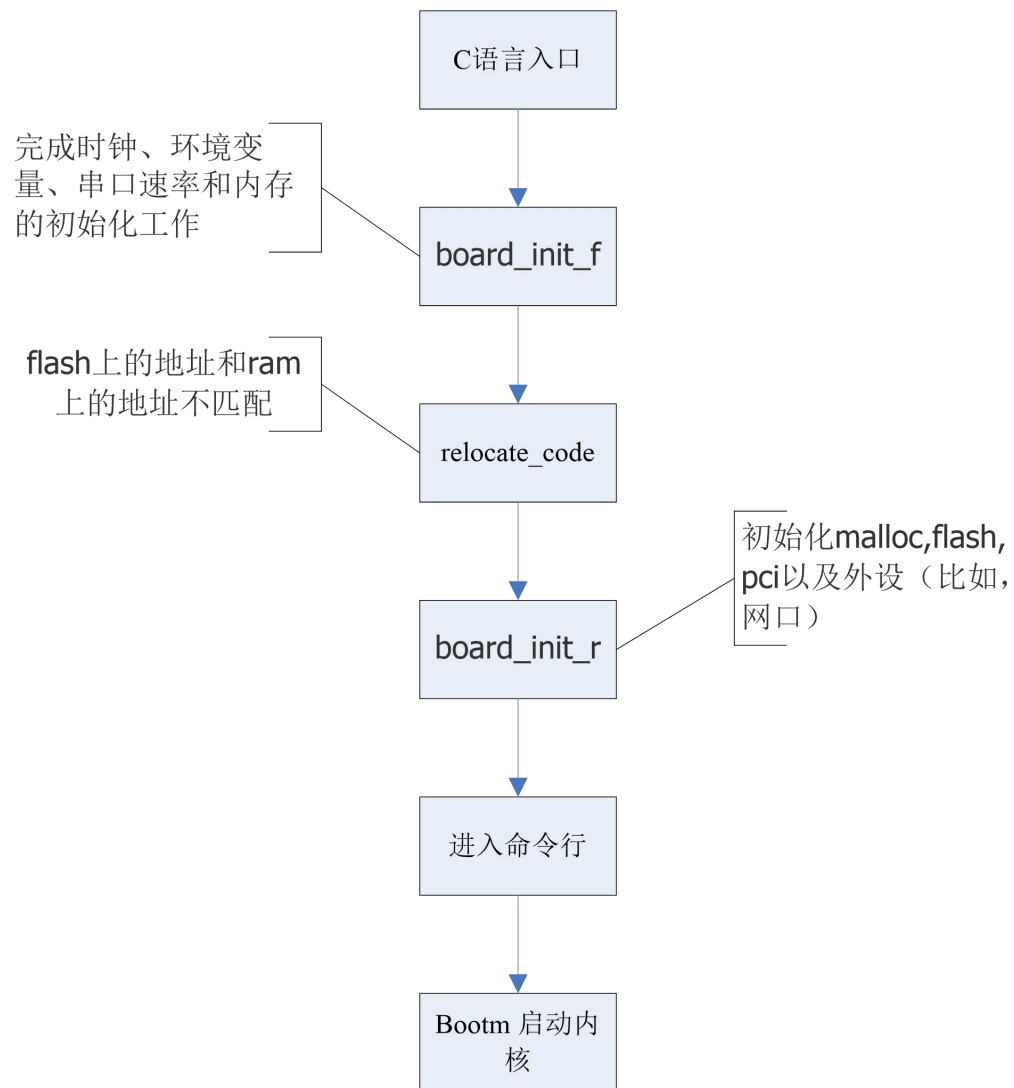
■ 调用board.c中的函数board_init_f做一系列初始化:

- timer_init 时钟初始化
- env_init 环境变量初始化 (取得环境变量存放的地址)
- init_baudrate 串口速率
- serial_init 串口初始化
- console_init_f 配置控制台
- display_banner 显示u-boot启动信息, 版本号等
- init_func_ram 初始化内存, 配置ddr controller



MIPS启动过程2 (C代码)

- 上述工作完成后，串口和内存都已经可以用了。然后进行内存划分，对堆和栈初始化，并留出u-boot代码大小的空间，把代码从flash上搬到ram上，继续执行
- 之后进入board.c的board_init_r函数，在这个函数里初始化flash, pci 以及外设（比如，网口），最后进入命令行或者直接启动Linux kernel





CPU仿真 (Emulation)

■ MOS在 QEMU 仿真器的启动

- QEMU已经提供了bootloader全部功能，MOS操作系统不需要再实现bootloader的功能
- 启动流程被简化为加载内核到内存，之后跳转到内核的入口

■ MOS 操作系统在 QEMU 仿真器上运行，QEMU 仿真器模拟了CPU的执行过程

- 利用C++等高层次语言模拟CPU执行，例如用变量模拟寄存器、控制结构模拟底层跳转等
- 类似但是不同于虚拟化 (例如 VMware, VirtualBox)，虚拟化直接在CPU上运行指令
- 操作系统启动只需要向仿真器中传入包含操作系统的磁盘镜像文件 (disk image file)



启动及OS引导

当我们打开计算机的电源开关，到我们看到OS的登录界面，计算机内部经历了怎样的过程？

■ MIPS CPU系统的启动

■ X86 CPU系统的启动

- 硬件初始化过程 (X86)
- Linux操作系统引导

MIPS处理器大多用于嵌入式系统，嵌入式系统常用U-boot作为OS启动装载程序，U-Boot(Universal Boot Loader)

X86处理器通常采用LILO和GRUB



X86 启动过程 (与OS无关)

1. Turn on
2. CPU jump to physical address of BIOS (0xFFFF0) (Intel 80386)
3. BIOS runs POST (Power-On Self Test)
4. Find bootable devices
5. Loads boot sector from MBR
6. BIOS yields control to OS BootLoader

第一阶段

第二阶段



BIOS (Basic Input/Output System)

- BIOS设置程序是被固化到电脑主板上地ROM芯片中的一组程序，其主要功能是为电脑提供最底层的、最直接的硬件设置和控制。BIOS通常与硬件系统集成在一起（在计算机主板的ROM或EEPROM中），所以也被称为**固件**



BIOS on board



BIOS on screen



BIOS

- BIOS程序存放于一个断电后内容不会丢失的只读存储器中；系统上电或被重置（reset）时，处理器要执行第一条指令的地址会被定位到BIOS的存储器中，让初始化程序开始运行。
- 在X86系统中，CPU加电后将跳转到BIOS的固定物理地址0xFFFF0。（Intel 80386）



BIOS和U-Boot的比较

特性	U-Boot	BIOS (x86)
架构	多种 (ARM, MIPS, PowerPC, x86 等)	x86
用途	嵌入式系统	桌面/服务器
开源	是	通常不是
可定制性	高	低
命令行界面	有	通常没有
网络功能	强大 (TFTP, DHCP, NFS, PXE)	有限 (通常只有 PXE)
大小	较小 (通常几百 KB 到几 MB)	较大 (通常几 MB 到几十 MB)



启动第一步——加载BIOS

- 当打开计算机电源，计算机会首先加载BIOS信息。BIOS中包含了CPU的相关信息、设备启动顺序信息、硬盘信息、内存信息、时钟信息、PnP特性等等。在此之后，计算机心里就有谱了，知道应该去读取哪个硬件设备了



- BIOS代码包含诊断功能，以保证某些重要硬件组件，像是键盘、磁盘设备、输出输入端口等等，可以正常运作且正确地初始化。几乎所有的BIOS都可以选择性地运行CMOS存储器的设置程序；也就是保存BIOS会访问的用户自定义设置数据（时间、日期、硬盘细节，等等）

```

Diskette Drive B : None
Serial Port(s) : 3F0 2F0
Pri. Master Disk : LBA,ATA 100, 250GB Parallel Port(s) : 370
Pri. Slave Disk : LBA,ATA 100, 250GB DDR at Bank(s) : 0 1 2
Sec. Master Disk : None
Sec. Slave Disk : None

Pri. Master Disk HDD S.M.A.R.T. capability ... Disabled
Pri. Slave Disk HDD S.M.A.R.T. capability ... Disabled

PCI Devices Listing ...

Bus Dev Fun Vendor Device SVID SSID Class Device Class IRQ
0 27 0 8086 2668 1458 A005 0403 Multimedia Device 5
0 29 0 8086 2658 1458 2658 0C03 USB 1.1 Host Cntrlr 9
0 29 1 8086 2659 1458 2659 0C03 USB 1.1 Host Cntrlr 11
0 29 2 8086 265A 1458 265A 0C03 USB 1.1 Host Cntrlr 11
0 29 3 8086 265B 1458 265A 0C03 USB 1.1 Host Cntrlr 5
0 29 7 8086 265C 1458 5006 0C03 USB 1.1 Host Cntrlr 9
0 31 2 8086 2651 1458 2651 0101 IDE Cntrlr 14
0 31 3 8086 266A 1458 266A 0C05 SMBus Cntrlr 11
1 0 0 10DE 0421 10DE 0479 0300 Display Cntrlr 5
2 0 0 1283 8212 0000 0000 0180 Mass Storage Cntrlr 10
2 5 0 11AB 4320 1458 E000 0200 Network Cntrlr 12
ACPI Controller 9

```



BIOS

读取启动顺序 (Boot Sequence)

- 现代的BIOS可以让用户选择由哪个设备引导电脑，如光盘驱动器、硬盘、软盘、USB U盘等等。这项功能对于安装操作系统、以CD引导电脑、以及改变电脑找寻开机媒体的顺序特别有用。

打开**BIOS**的操作界面，里面有一项就是"设定启动顺序"。





BIOS的问题

- 16位~20位实模式寻址能力（问：地址空间？）

- 实现结构、可移植性

- 问题根源

- 历史的局限性、向前兼容的压力

- 支持遗留软件：老设备驱动等

- 经典≈（成熟、稳定、共识），来之不易，维持整个产业生态正常运转的必要Tradeoff

- IT发展太快，对“历史局限”的继承，导致改变成本越来越高。——“另起炉灶”（UEFI）来解决



UEFI——统一可扩展固件接口



■ Unified Extensible Firmware Interface

- 2000年提出，Intel组建生态

■ 功能特性

- 支持从超过2TB的大容量硬盘引导 (GUID Partition Table, GPT分区) (硬件支持)
- CPU-independent architecture(可移植性)
- CPU-independent drivers (可移植性)
- Flexible pre-OS environment, including network capability (硬件支持)
- Modular design (可移植性)



UEFI和BIOS的比较

二者显著的区别是：

- EFI是用模块化，**C语言**风格的参数堆栈传递方式，动态链接的形式构建的系统，较BIOS而言更易于实现，容错和纠错特性更强，缩短了系统研发的时间。
- 它运行于32位或64位模式，乃至未来增强的处理器模式下，突破传统BIOS的16位代码的**寻址能力**，达到处理器的最大寻址。





启动第二步——读取MBR

- 硬盘上第0磁头第0磁道第一个扇区被称为MBR，也就是Master Boot Record，即主引导记录，它的大小是512字节，别看地方不大，可里面却存放了预启动信息、分区表信息。



磁盘小知识

■ 扇区 (sector)

- 盘片被分成许多扇形的区域

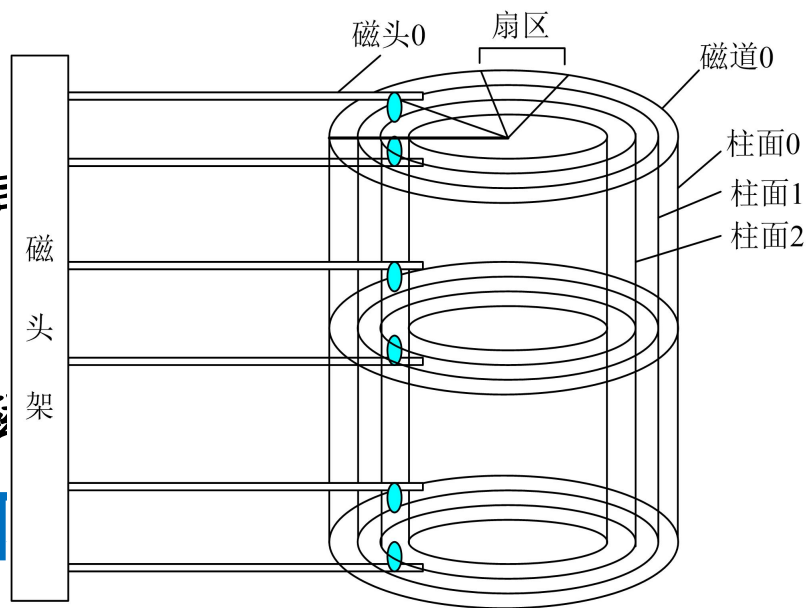
■ 磁道 (track)

- 盘片上以盘片中心为圆心，不同

■ 柱面 (cylinder)

- 硬盘中，不同盘片相同半径的磁道

■ 每个磁盘有两个面，每个面都





装入MBR

- MBR的全称是Master Boot Record (主引导记录), MBR早在1983年IBM PC DOS 2.0中就已经提出。之所以叫“主引导记录”, 是因为它是存在于驱动器开始部分的一个特殊的启动扇区。
- 这个扇区包含了已安装的操作系统的启动加载器(BootLoader)和驱动器的逻辑分区信息。
- MBR 是一个512-byte的扇区, 位于磁盘的固定位置 (sector 1 of cylinder 0, head 0)

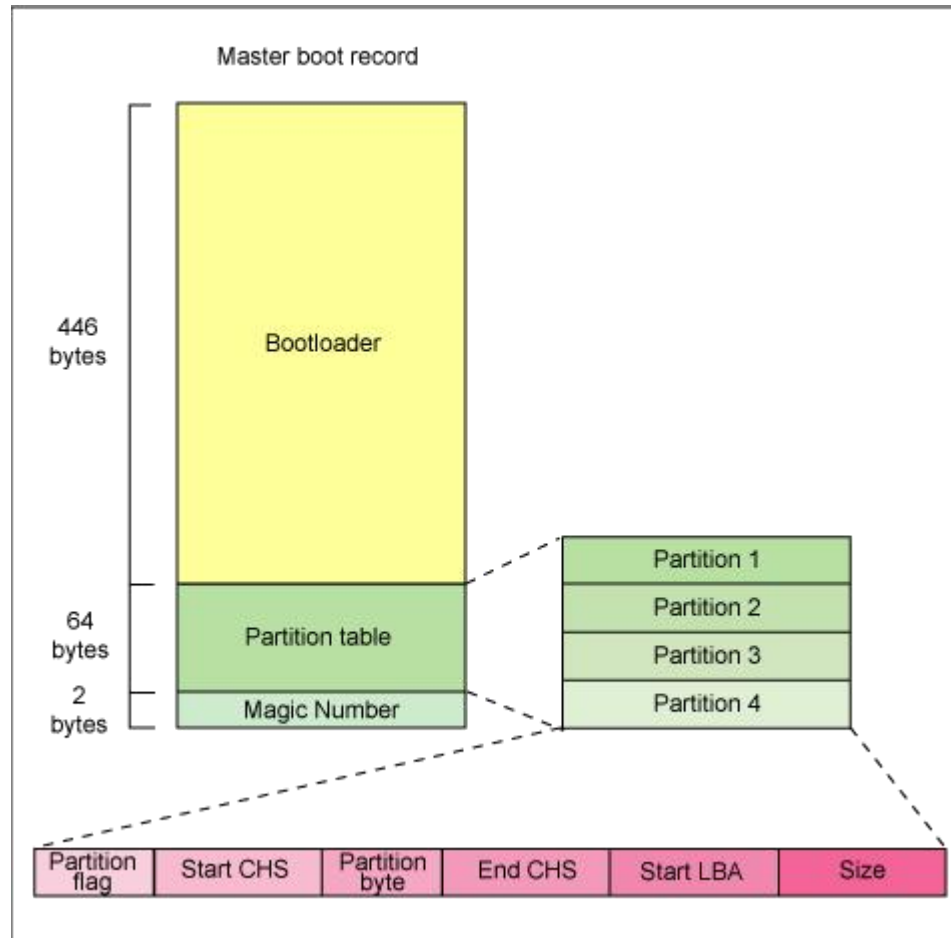


MBR的结构

- MBR(Master Boot Record)主引导记录包含两部分的内容，前446字节为启动代码及数据
- 之后则是分区表 (DPT, Disk Partition Table) , 分区表由四个分区项组成，每个分区项数据为16字节，记录了启动时需要的分区参数。这64个字节分布在MBR的第447-510字节
- 后面紧接着两个字节AA和55被称为幻数(Magic Number), BIOS读取MBR的时候总是检查最后是不是有这两个幻数,如果没有就被认为是一个没有被分区的硬盘。



MBR (Master Boot Record)





MBR

Address		Description		Size (bytes)
Hex	Dec			
+0x0000	+0	Bootstrap code area		446
+0x01BE	+446	Partition entry #1	<i>Partition table</i> (for primary partitions)	16
+0x01CE	+462	Partition entry #2		16
+0x01DE	+478	Partition entry #3		16
+0x01EE	+494	Partition entry #4		16
+0x01FE	+510	55h	<i>Boot signature</i>	2
+0x01FF	+511	AAh		
Total size: $446 + 4 \times 16 + 2$				512

存储字节位	内容及含义
第1字节	引导标志。若值为80H表示活动分区，若值为00H表示非活动分区。
第2、3、4字节	本分区的起始磁头号、扇区号、柱面号。其中： 磁头号——第2字节； 扇区号——第3字节的低6位； 柱面号——为第3字节高2位+第4字节8位。
第5字节	分区类型符。 00H——表示该分区未用（即没有指定）； 06H——FAT16基本分区； 0BH——FAT32基本分区； 05H——扩展分区； 07H——NTFS分区； 0FH——（LBA模式）扩展分区（83H为Linux分区等）。
第6、7、8字节	本分区的结束磁头号、扇区号、柱面号。其中： 磁头号——第6字节； 扇区号——第7字节的低6位； 柱面号——第7字节的高2位+第8字节。
第9、10、11、12字节	本分区之前已用了的扇区数。
第13,14,15,16字节	本分区的总扇区数。



MBR

- 由于MBR的限制 只能有4个主分区，系统必须装在主分区上面。
- 硬盘分区有三种，主磁盘分区、扩展磁盘分区、逻辑分区。
- 一个硬盘主分区至少有1个，最多4个，扩展分区可以没有，最多1个。且主分区+扩展分区总共不能超过4个。逻辑分区可以有若干个。
- 主分区只能有一个是激活的（active），其余为inactive。



启动及OS引导

当我们打开计算机的电源开关，到我们看到OS的登录界面，计算机内部经历了怎样的过程？

■ MIPS CPU系统的启动

■ X86 CPU系统的启动

- 硬件初始化过程 (X86)
- Linux操作系统引导

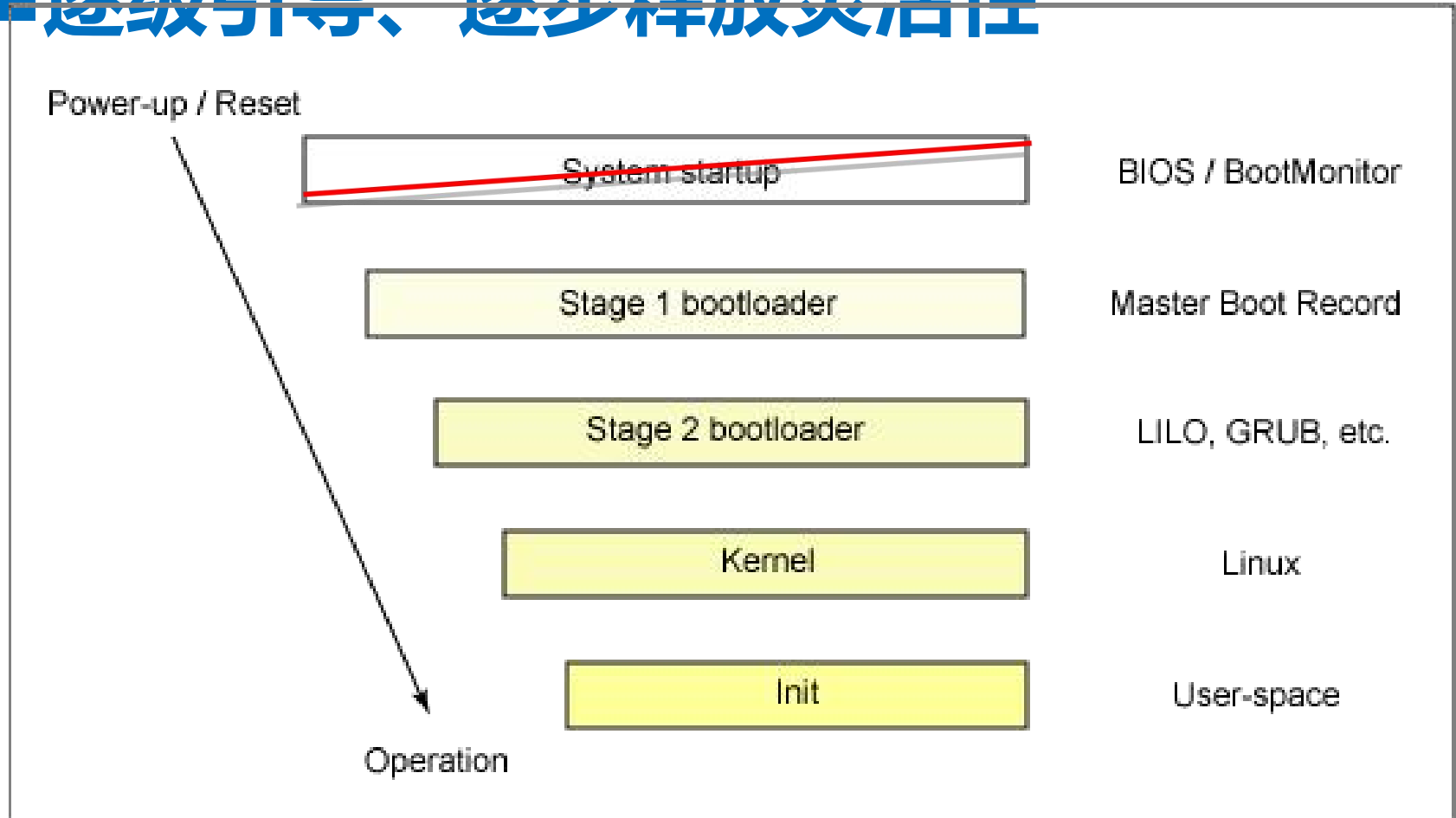
MIPS处理器大多用于嵌入式系统，嵌入式系统常用U-boot作为OS启动装载程序，U-Boot(Universal Boot Loader)

X86处理器通常采用LILO和GRUB



How Linux boot?

■ 逐级引导、逐步释放灵活性





启动第三步——Boot Loader

- **Boot Loader** 就是在操作系统内核运行之前运行的一段小程序。通过这段小程序，可以初始化硬件设备、建立内存空间的映射图，从而将系统的软硬件环境带到一个合适的状态，以便为最终调用操作系统内核做好一切准备



Boot loader

- **Boot loader 也可以称之为操作系统内核加载器(OS kernel loader), 是操作系统内核运行之前运行的一段小程序。通过这段小程序, 我们可以初始化硬件设备、建立内存空间的映射图, 从而将系统的软硬件环境带到一个合适的状态, 以便为最终调用操作系统内核做好一切准备。通常是严重地依赖于硬件而实现的。**
- **GRUB 和 LILO 最重要的Linux加载器。**
 - **Linux Loader (LILO)**
 - **GRand Unified Bootloader (GRUB)**



LILO: Linux LOader

- *Linux LOader (LILO)* 已经成为所有 Linux 发行版的标准组成部分, 是最老的 Linux 引导加载程序。
- LILO的主要优点是, 它可以快速启动安装在主启动记录中的 Linux操作系统。
- LILO的主要局限是, LILO 配置文件被反复更改时, 主启动记录 (MBR) 也需要反复重写, 但重写可能发生错误, 这将导致系统无法引导。



GNU GRUB

■ GNU GRand Unified Bootloader

- 允许用户可以在计算机内同时拥有多个操作系统，并在计算机启动时选择希望运行的操作系统。

```
GNU GRUB  version 0.97  (637K lower / 1046400K upper memory)
```

```
Red Hat Enterprise Linux (2.6.32-279.el6.x86_64)
Windows XP SP3
```

```
Use the ↑ and ↓ keys to select which entry is highlighted.
Press enter to boot the selected OS, 'e' to edit the
commands before booting, 'a' to modify the kernel arguments
before booting, or 'c' for a command-line.
```



GRUB 与 LILO 的比较

- LILO 没有交互式命令界面，而 GRUB 拥有。
- LILO 不支持网络引导，而 GRUB 支持。
- LILO 将关于可以引导的操作系统位置的信息物理上存储在 MBR 中。如果修改了 LILO 配置文件，必须将 LILO 第一阶段引导加载程序重写到 MBR。错误配置的 MBR 可能会让系统无法引导。
- 使用 GRUB，如果配置文件配置错误，则只是默认转到 GRUB 命令行界面。



GRUB磁盘引导过程

- **stage1: grub读取磁盘第一个512字节**（硬盘的0道0面1扇区，被称为**MBR**（主引导记录），也称为bootsect）。MBR由一部分bootloader的引导代码、分区表和魔数三部分组成。（**启动的第二步**）
- **Stage1.5: 识别各种不同的文件系统格式**。这使得grub识别到文件系统。
- **stage2: 加载系统引导菜单**(/boot/grub/menu.lst或grub.lst)，**加载内核映像**(kernel image)和RAM磁盘initrd（可选）。



运行主引导程序

■ BIOS将硬盘的主引导记录（位于0柱面、0磁道、1扇区）读入7C00处，然后将控制权交给主引导程序（GRUB的第一阶段）。任务包括：

1. 检查（WORD）0x7dfe是否等于0xaa55。若不等于则转去尝试其他介质；如果没有其他启动介质，则显示 “No ROM BASIC” ，然后死机；
2. 跳转到0x7c00处执行MBR中的程序；
3. 将自己复制到0x0600处，然后继续执行；

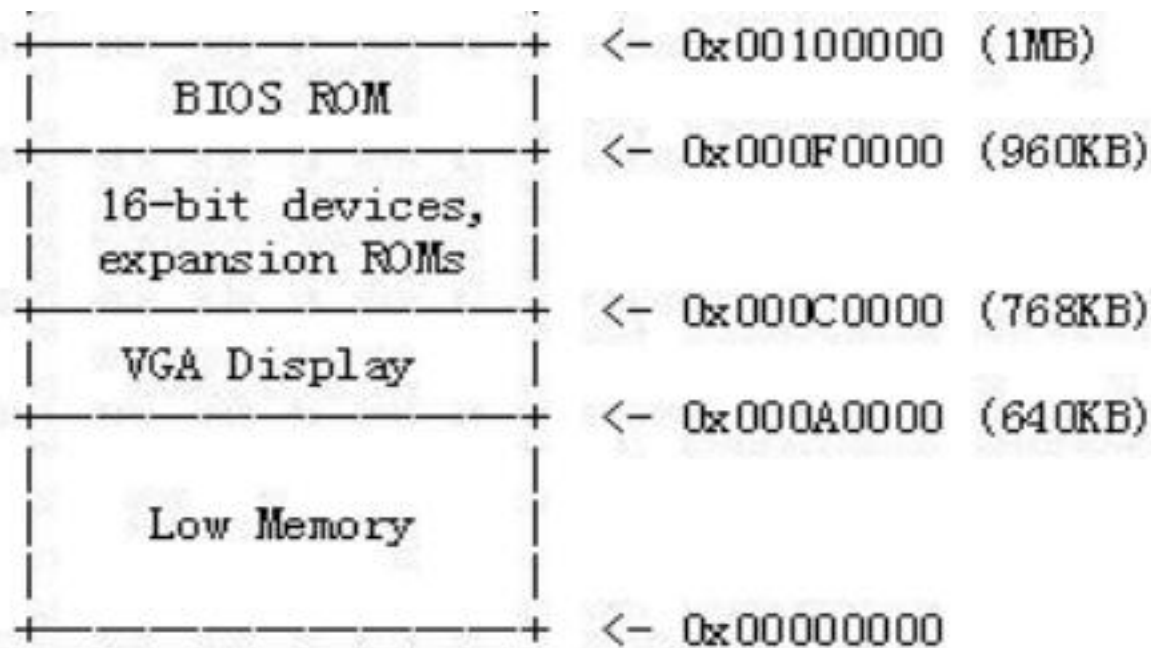


运行主引导程序

4. 在主分区表中搜索标志为活动的分区。如果发现没有活动分区或者不止一个活动分区，则停止；
5. 将活动分区的第一个扇区读入内存地址0x7c00处；
6. 检查位于地址0x7dfe的（WORD内容）是否等于0xaa55，若不等于则显示 “Missing Operating System”，然后停止，或尝试软盘启动；
7. 跳转到0x7c00处继续执行特定系统的启动程序；



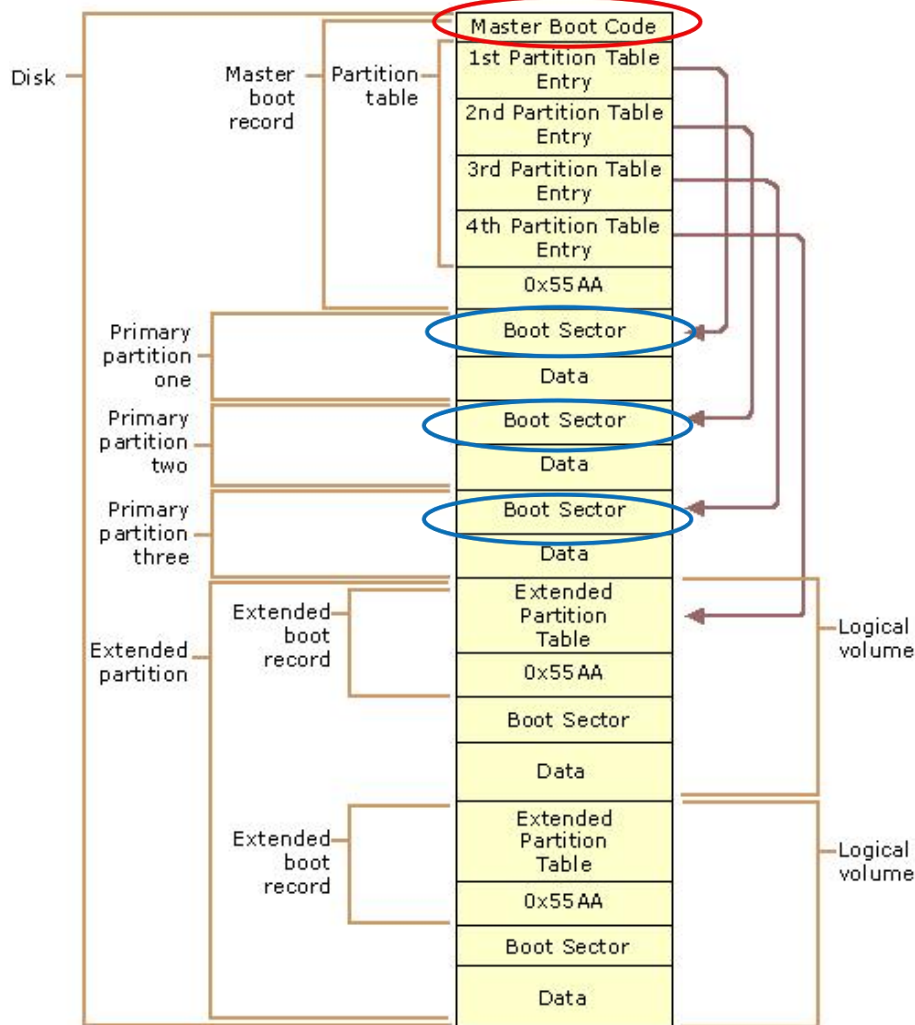
- PC机最早是由IBM生产，使用的是Intel 8088处理器。这个处理器只有20根地址线，可以寻址1M的空间。这1M空间大概有如下的结构：





MBR与引导扇区Boot Sector 的关系

- MBR存放的位置是整个硬盘第一个扇区
- Boot Sector是硬盘上每个分区的第一个扇区





Kernel image

- The kernel is the central part in most computer operating systems because of its task, which is the **management of the system's resources and the communication between hardware and software components.**
- Kernel is **always stored on memory** until computer is turn off.
- Kernel image is not an executable kernel, but a **compress kernel image.**
 - zImage size less than 512 KB
 - bzImage size greater than 512 KB

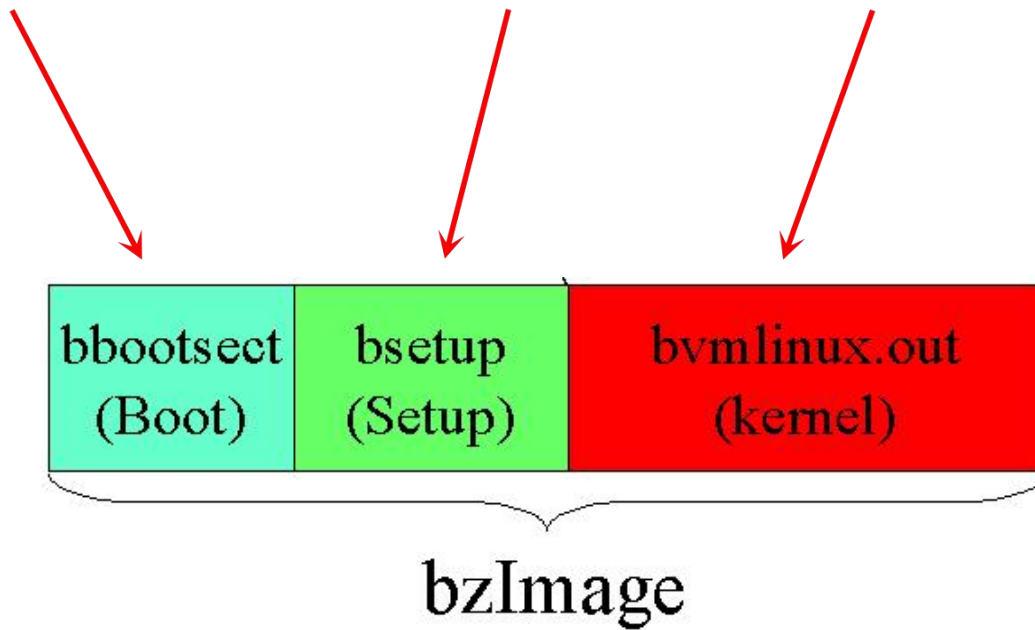


Kernel Image

引导代码
Bootsect.s

启动代码
Setup.s

压缩的
OS代码





Task of kernel

■ 资源管理

- Process management
- Memory management
- Device management

■ 用户服务

- System call



启动第四步——加载内核

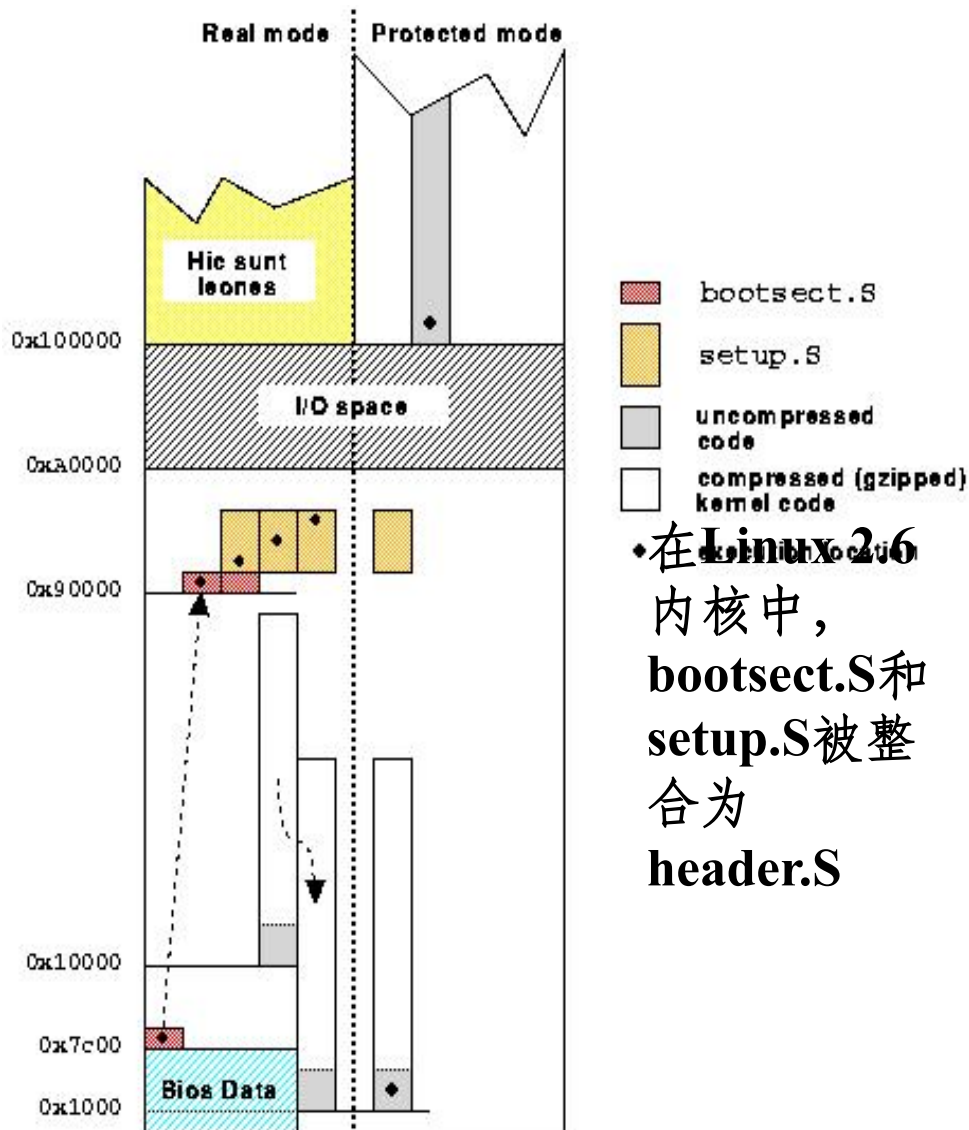
- 根据grub设定的内核映像所在路径，系统读取内存映像，并进行解压缩操作。
- 系统将解压后的内核放置在内存之中，初始化函数并初始化各种设备，完成Linux核心环境的建立。



Linux Kernel加载过程

■ 不断装载下一段可执行代码

- 扇区拷贝
- 支持文件系统
- 设置内存
- 解压缩
- 切换CPU模式
- ...



• 在Linux 2.6内核中，bootsect.S和setup.S被整合为header.S

<https://en.wikipedia.org/wiki/Vmlinux>



Bootsect.s

- ① boot过程首先将自身从原始启动区0x7c00—0x7dff移至0x90000—0x901ff,并跳至下一条指令。
- ② 读引导扇区的后4个扇区到0x90200(即setup区的头4个扇区)。设置一些参数: 如堆栈(栈基址9000:3ff4). 新的磁盘参数表等
- ③ 加载内核映像: 如果是大内核, 加载到0x100000 (1M以上高端内存), 否则加载到0x010000 (64K低端内存)。
- 跳转到Setup的入口点 (即转到地址为0x90020的地址处执行, 也就是开始执行setup部分的代码了)。





几个小问题

■ 为什么需要磁盘引导程序

- 为什么需要磁盘引导程序而不是直接载入操作系统？或许是因为历史原因，BIOS最初是为最原始的8位PC所创，8位的PC磁盘很小，所以BIOS只能先载入磁盘引导程序然后通过磁盘引导程序导入操作系统。

■ 磁盘引导程序为什么要从1扇区，0磁头，0磁道加载到7c00H处

- “引导程序加载器--int19”。由于BIOS加载磁盘引导程序时需要调用系统中断int19来加载引导程序，而这个中断指定了将磁盘1扇区，0磁头，0磁道加载到7c00H处。

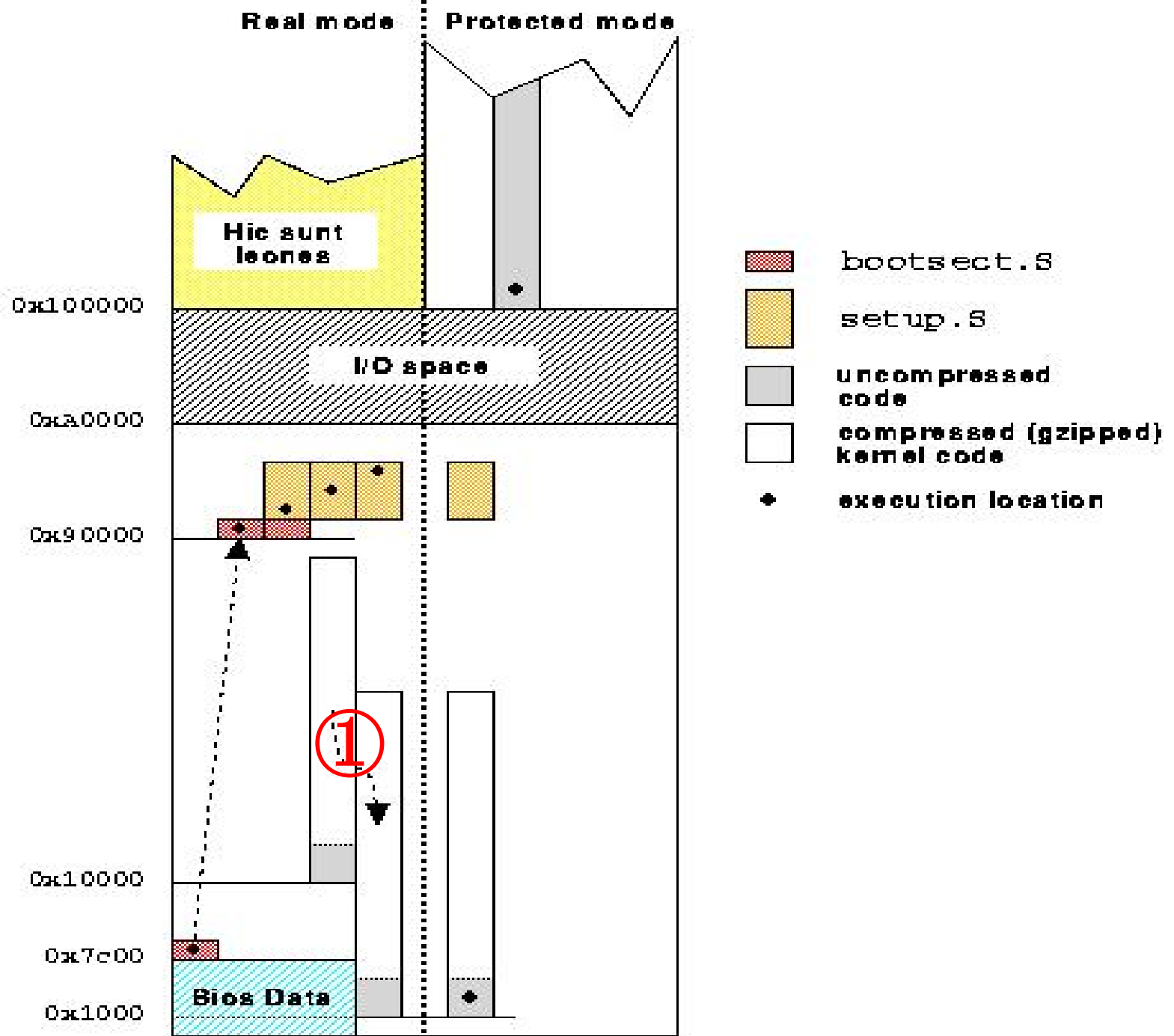
■ 磁盘引导程序为什么是16位

- 因为DOS系统是16位，X86系统都向后兼容引导时的16位模式。



Setup.S

- 初始化硬件设备。如：调用BIOS例程建立描述系统物理内存布局的表；设置键盘的重复延迟和速率；初始化显卡；检测IBM MCA总线、PS/2鼠标设备、APM BIOS支持等.....。
- ① 如果内核加载在低RAM地址0x00010000，则把它移动到0x00001000处；如果映像加载在高内存1M位置，则不动；
- 为内核程序的执行建立环境。如：启动位于8042键盘控制器的A20 pin。建立一个中断描述表IDT和全局描述表GDT表；重启FPU单元；对可编程中断控制器进行重新编程，屏蔽所有中断；设置CR0状态寄存器的PE位使CPU从实模式切换到保护模式，PG位清0，禁止分页功能等.....；
- 跳转到startup_32()汇编函数, jmp `0x100000, __BOOT_CS`, 终于进入内核Head.S；

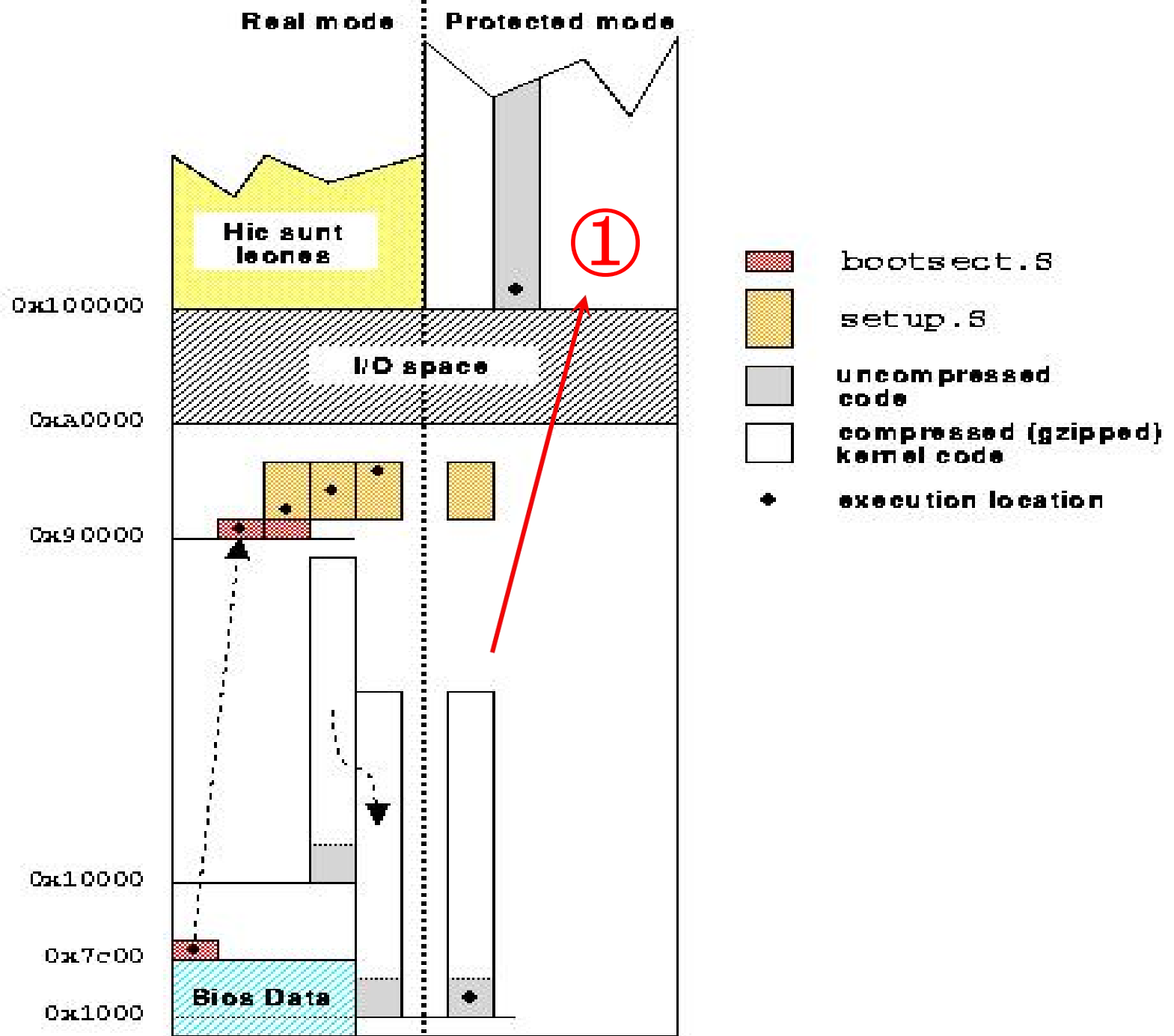




Head.S (第一个start_32()函数)

setup结束后，该函数被放在0x00001000或者0x00100000位置，该函数主要操作：

- 首先初始化段寄存器和临时堆栈；
- 清除eflags寄存器的所有位；
- 将_edata和_end区间的所有内核未初始化区填充0；
- ① 调用decompress_kernel()函数解压内核映像。首先显示"Uncompressing Linux..."信息，解压完成后显示"OK, booting the kernel."。内核解压后，如果时低地址载入，则放在0x00100000位置；否则解压后的映像先放在压缩映像后的临时缓存里，最后解压后的映像被放置到物理位置0x00100000处；
- 跳转到0x00100000物理内存处执行；





Head.s (第二个start_32()函数)

■ 解压后的映像开始于arch/i386/kernel/head.S 文件中的 startup_32()函数，因为通过物理地址的跳转执行该函数的，所以相同的函数名并没有什么问题。该函数为Linux第一个进程建立(x86)执行环境，操作如下：

- a) 初始化ds,es,fs,gs段寄存器的最终值；
- b) 用0填充内核bss段；
- c) 初始化swapper_pg_dir数组和pg0包含的临时内核页表
- d) 建立进程0idle进程的内核模式的堆栈；

.....

- x) 跳转到start_kernel函数，这个函数是第一个C编制的函数，内核又有了一个新的开始。



调用Start_kernel()

■ 调用start_kernel()函数来启动一系列的初始化函数并初始化核心数据结构和各种设备，完成Linux核心环境的建立：

- 调度器初始化，调用sched_init();
- 调用build_all_zonelists函数初始化内存区；
- 调用page_alloc_init()和mem_init()初始化伙伴系统分配器；
- 调用trap_init()和init_IRQ()对中断控制表IDT进行最后的初始化；
- 调用softirq_init() 初始化TASKLET_SOFTIRQ和HI_SOFTIRQ；
- Time_init()对系统日期和时间进行初始化；
- 调用kmem_cache_init()初始化slab分配器；
- 调用calibrate_delay()计算CPU时钟频率；

至此，Linux内核已经建立起来了，基于Linux的程序应该可以正常运行了。



启动第五步——用户层init依据inittab文件来设定运行等级

- 内核被加载后，第一个运行的程序便是/sbin/init，该文件会读取/etc/inittab文件，并依据此文件来进行初始化工作。
- /etc/inittab文件最主要的作用就是设定Linux的运行等级，其设定形式是“: id:5:initdefault:”，这就表明Linux需要运行在等级5上。



Linux的运行等级设定如下：

- 0： 关机
- 1： 单用户模式
- 2： 无网络支持的多用户模式
- 3： 有网络支持的多用户模式
- 4： 保留， 未使用
- 5： 有网络支持有X-Window支持的多用户模式
- 6： 重新引导系统， 即重启



启动第六步 - - init进程执行rc.sysinit

- 在设定了运行等级后，Linux系统执行的第一个用户层文件就是/etc/rc.d/rc.sysinit脚本程序，它做的工作非常多，包括设定PATH、设定网络配置（/etc/sysconfig/network）、启动swap分区、设定/proc等等。
- 如果你有兴趣，可以到/etc/rc.d中查看一下rc.sysinit文件，里面的脚本够你看的😊



启动第七步 - - 启动内核模块

- 具体是依据 `/etc/modules.conf` 文件或 `/etc/modules.d` 目录下的文件来装载内核模块。



启动第八步 - - 执行不同运行级别的脚本程序

- 根据运行级别的不同，系统会运行rc0.d到rc6.d中的相应的脚本程序，来完成相应的初始化工作和启动相应的服务。



启动第九步 - - 执行/etc/rc.d/rc.local

- 你如果打开了此文件，里面有一句话，读过之后，你就会对此命令的作用一目了然：

This script will be executed **after all the other init scripts.**

You can put your own initialization stuff in here if you don't

want to do the full Sys V style init stuff.

- **rc.local就是在一切初始化工作后，Linux留给用户进行个性化的地方。你可以把你想要设置和启动的东西放到这里。**



启动第十步 - - 执行/bin/login程序， 进入登录状态

- 此时，系统已经进入到了等待用户输入username和password的时候了，你已经可以用自己的帐号登入系统了。



漫长的启动过程结束了，一切都清静了.....

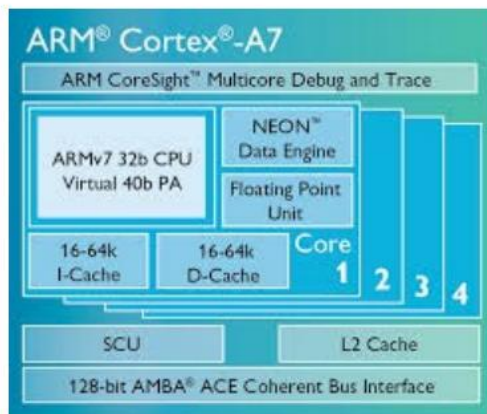
其实在这背后，还有着更加复杂的底层函数调用，等待着你去研究.....

你能否从这些复杂的启动过程中总结出一个最简单的启动需要哪些功能？



结合实验

- 阅读《See MIPS Run》，了解MIPS启动过程
- 查阅资料，了解ARM（树莓派）启动过程





思考：

- 为什么操作系统启动慢？给出对OS启动过程的分析，指出最耗时的启动过程是什么？现有的优化措施有哪些？
- 多核CPU的操作系统如何启动，与单核CPU操作系统主要区别在哪里？